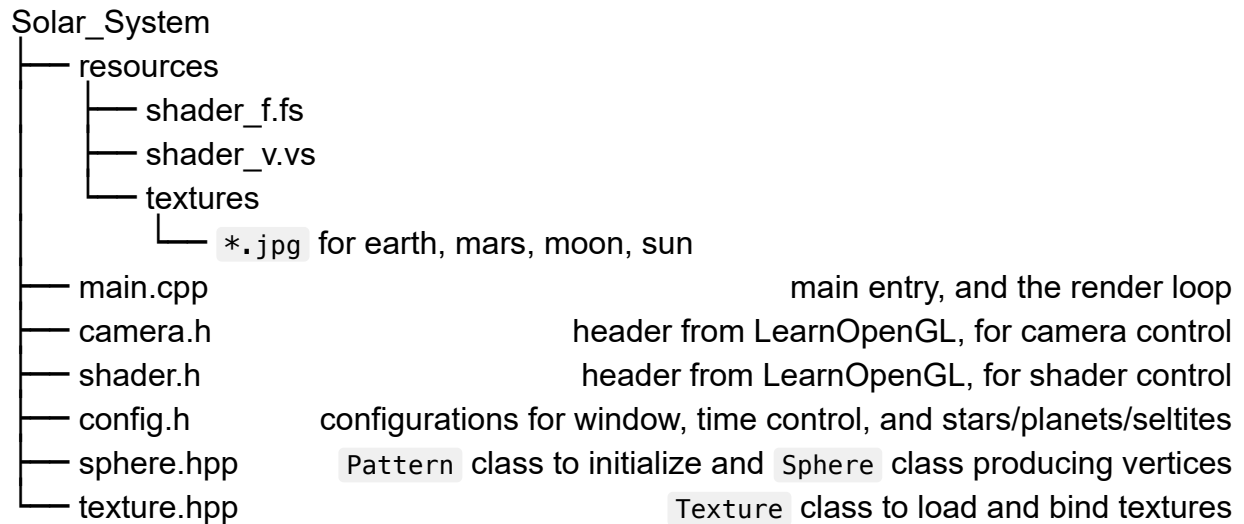


### Source File Structure (briefly description)



### Credits

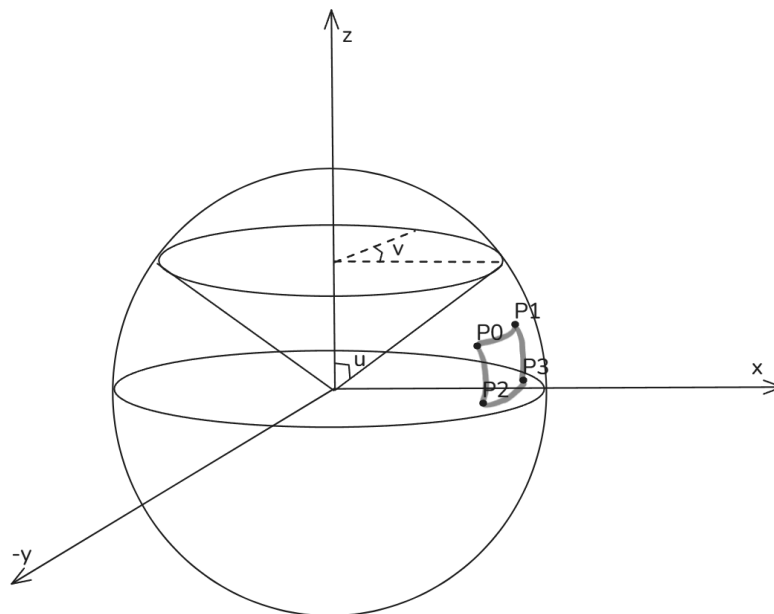
- Textures from <https://www.solarsystemscope.com/textures/>
- Some header files from <https://learnopengl-cn.github.io/> (with a little modification)
- Others are implemented by myself purely

- in `main.cpp`

```
1  /* include files */
2
3  /* initialize camera and time */
4
5  /* functions for interaction */
6
7  int main {
8      /* initialize window and OpenGL */
9
10     /* initialize one shader(and settings), textures and one sphere */
11
12     while (!glfwWindowShouldClose(window)) {
13         /* process input, clear window and update camera */
14
15         // render sun
16         // render earth
17         // render moon
18         // render mars
19     }
20     /* release resources */
21 }
```

- `shader.h` and `camera.h` are from LearnOpenGL. Only add the `UP` and `DOWN` direction for camera control in `camera.h`

- `config.h` is a header file for configurations, including
  - window size, time control
    - I treat unit of `glfwGetTime()` as hours, so there is an global speeding up ratio
  - stars/planets/selites settings, including **radius(scale)**, **distance**, **RotationAngle**, **RotationSpeed**, **revolution speed**
- `sphere.hpp`
  - One class `Pattern` to **initialize** and bind **VAO**, **VBO**, **EBO** for any pattern, and **designed to**
    - **adapt to different shaders**, such as
      1. A vertex with color in respective RGB format
      2. A vertex with color in uniform RGB format
      3. A vertex with texture coordinate
    - **adapt to cases where vertices have been created or not** (create later)
    - **adapt to cases where using indices or not**
    - **Convenient**: once created, just bind shader (and texture) and call `draw()` for rendering
  - One class `Sphere` inherit from `Pattern` to construct vertices and indices
    - according to the **radius**, **Longitude** and **Latitude Resolution** (they are adjustable)
    - $u \in [0, 180]$ ,  $v \in [0, 360]$ , create two triangles at each position, like the following



- `texture.hpp`: one class to load textures and return the texture ID, nothing special

- How to navigate in the space?
  - W, A, S, D for moving **forward**, **left**, **backward**, **right**
  - SPACE for moving **up**, LEFT SHIFT + SPACE for moving **down**
  - use **mouse** to control the **view direction**
- compile commandline:
  - `g++ -std=c++17 -Wall -Wextra -g -Iinclude -o output/main.exe src/main.cpp -Llib -lglad -lglfw3dll -llibassimp -lopengl32 -lglu32`
  - `./output/main.exe` with `glfw3.dll`, `libassimp-5.dll` in the same directory
  - Maybe need some specific lib or include requirements, and dynamic link libraries

- How is the **rotation** and **revolution** implemented
  - For each star/planet/selite, use a `glm::mat4` to store the model matrix
  - **explicitly** compute the **revolution** by `sin`, `cos` and `dis`, use `translate` to move
  - use `rotate` to **tilt** the star/planet/selite **on its axis** according to the angle, and then `rotate` **around the axis**
  - use `scale` to scale the star/planet/selite
  - For all the stars/planets/selites, they are rendered by changing **only the model matrix** and **texture**. The sphere is **reused**
- For more details, please refer to the **source code** and the **comments**

### ✓ Check

- ✓ -2 suns, 2+ planets, 1+ satellite
  - Sun, Earth, Mars, Moon
  - an additional smaller sun in a casual position
- ✓ Planets orbit around the sun
  - Earth, Mars around the Sun
- ✓ Satellites orbit around its planet
  - Moon around the Earth
- ✓ Trajectories are not co-planar
  - each sphere rotate around its axis
- ✓ Navigation in the system (3D viewing)

- Result like this

